

Zaawansowane techniki analizy skupień: praktyczne ćwiczenia

1. Wprowadzenie teoretyczne

Analiza skupień (ang. *cluster analysis*) to technika eksploracyjna wykorzystywana do grupowania obiektów w zbiory (skupienia), w taki sposób, by obiekty w tej samej grupie były do siebie bardziej podobne niż obiekty z innych grup.

1.1 Miary podobieństwa

Do oceny podobieństwa pomiędzy obiektami często stosuje się miary odległości:

- **Euklidesowa:** $d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$
- **Manhattan:** $d(x, y) = \sum_{i=1}^n |x_i - y_i|$
- **Cosinusowa:** $d(x, y) = 1 - \frac{x \cdot y}{\|x\| \|y\|}$

1.2 Algorytmy grupowania

W analizie skupień stosuje się różne algorytmy, które różnią się podejściem do formowania grup. Poniżej opisano najważniejsze z nich:

- **K-means:** Klasyczny i najczęściej stosowany algorytm centroidowy. Działa iteracyjnie, dążąc do minimalizacji wariancji wewnątrz skupień. Dla zadanej liczby k skupień, algorytm:
 1. Losowo inicjalizuje k centroidów,

2. Przypisuje każdy punkt do najbliższego centroidu (minimalizując odległość euklidesową),
3. Oblicza nowe centroidy jako średnie punktów w każdym skupieniu,
4. Powtarza kroki 2–3 aż do zbieżności (brak zmian w przypisaniach lub centroidach).

Wadą metody jest wrażliwość na wartości początkowe oraz konieczność wcześniejszego ustalenia liczby skupień.

- **DBSCAN (Density-Based Spatial Clustering of Applications with Noise):** Algorytm oparty na gęstości, który identyfikuje skupienia jako gęsto zaludnione regiony przestrzeni. Umożliwia wykrywanie skupień o nieregularnych kształtach oraz dobrze radzi sobie z szumem. Wymaga dwóch parametrów:

- `eps` – promień sąsiedztwa,
- `min_samples` – minimalna liczba punktów w sąsiedztwie, aby punkt uznać za rdzeniowy.

Punkty klasyfikowane są jako:

- *rdzeniowe* (gęste),
- *brzegowe* (w sąsiedztwie rdzeniowych),
- *szum* (nie należą do żadnego skupienia).

- **Agglomerative Clustering:** Hierarchiczna metoda łączenia skupień. Zaczyna się od traktowania każdego punktu jako oddzielnego skupienia, a następnie łączy je według określonego kryterium odległości:

- *najbliższy sąsiad* (single linkage),
- *najdalszy sąsiad* (complete linkage),
- *średnia odległość* (average linkage),
- *odległość centroidów*.

Proces trwa do momentu osiągnięcia zadanej liczby skupień lub jednej grupy nadrzędnej. Efekt działania algorytmu można przedstawić w postaci dendrogramu.

1.3 Wybór liczby skupień

Wybór odpowiedniej liczby skupień jest kluczowym krokiem w analizie skupień, ponieważ zbyt mała liczba skupień może prowadzić do nadmiernej agregacji danych, natomiast zbyt duża może prowadzić do nadmiernego rozbicia danych i przeuczenia. W praktyce stosuje się kilka metod do oszacowania optymalnej liczby skupień:

- **Metoda łokcia:** Polega na analizie wykresu zależności sumy kwadratów błędów (ang. *inertia*) od liczby skupień k . Wartość ta określana jest jako:

$$W(k) = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$

gdzie C_i to i -te skupienie, a μ_i to jego centroid. Wybieramy taką wartość k , dla której dalsze zwiększanie liczby skupień nie powoduje już znacznego spadku $W(k)$ (miejsce "łokcia" na wykresie).

- **Wskaźnik Silhouette:** Miara ta łączy informacje o spójności wewnątrz skupienia oraz separacji pomiędzy skupieniami. Dla każdego punktu i wartość wskaźnika oblicza się jako:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

gdzie:

- $a(i)$ - średnia odległość punktu do wszystkich innych punktów w tym samym skupieniu (spójność),
- $b(i)$ - najmniejsza średnia odległość do punktów z innego skupienia (separacja).

Wartość wskaźnika mieści się w przedziale $[-1, 1]$. Wartości bliskie 1 oznaczają dobrą jakość skupienia, wartości bliskie 0 sugerują nakładanie się skupień, a wartości ujemne wskazują, że punkt został przypisany do niewłaściwego skupienia.

- **Kryterium Calinskiego-Harabasza (CH):** Definiowane jako stosunek wariancji między skupieniami do wariancji wewnątrz skupień:

$$CH = \frac{\text{Tr}(B_k)}{\text{Tr}(W_k)} \cdot \frac{n - k}{k - 1}$$

gdzie:

- $\text{Tr}(B_k)$ - ślad macierzy wariancji między skupieniami,

- $\text{Tr}(W_k)$ - ślad macierzy wariancji wewnątrz skupień,
- n - liczba obserwacji, k - liczba skupień.

Im większa wartość CH, tym lepszy podział danych.

- **Kryterium Daviesa-Bouldina (DB):** Ocenia przeciętną "podobność" pomiędzy każdą parą skupień:

$$DB = \frac{1}{k} \sum_{i=1}^k \max_{j \neq i} \left(\frac{s_i + s_j}{d_{ij}} \right)$$

gdzie:

- s_i - średnia odległość punktów w skupieniu i do jego centroidu,
- d_{ij} - odległość między centroidami skupień i i j .

Niższe wartości wskaźnika DB oznaczają lepsze skupienia.

2. Praktyczne ćwiczenia w Pythonie

W tej części przeprowadzimy praktyczne eksperymenty z analizą skupień, używając bibliotek `scikit-learn`, `matplotlib` oraz `seaborn`. Wykorzystamy różne algorytmy oraz metody oceny liczby skupień.

2.1 Przygotowanie danych

```
1 from sklearn.datasets import make_blobs
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4
5 X, y = make_blobs(n_samples=300, centers=4, cluster_std=0.60, random_state=0)
6 plt.scatter(X[:, 0], X[:, 1], s=30)
7 plt.title("Wygenerowane dane")
8 plt.show()
```

2.2 K-means i metoda łokcia

```
1 from sklearn.cluster import KMeans
2 inertia = []
3 k_values = range(1, 11)
4
5 for k in k_values:
6     kmeans = KMeans(n_clusters=k, random_state=0)
7     kmeans.fit(X)
8     inertia.append(kmeans.inertia_)
9
10 plt.plot(k_values, inertia, marker='o')
11 plt.xlabel("Liczba skupie ")
12 plt.ylabel("Inertia")
13 plt.title("Metoda okcia ")
14 plt.show()
```

2.3 Wskaźnik Silhouette dla K-means

```
1 from sklearn.metrics import silhouette_score
2
3 kmeans = KMeans(n_clusters=4, random_state=0)
4 y_kmeans = kmeans.fit_predict(X)
5
6 score = silhouette_score(X, y_kmeans)
7 print(f"Wska nik Silhouette dla K-means: {score:.2f}")
```

2.4 DBSCAN i dobór parametrów

```
1 from sklearn.cluster import DBSCAN
2 import numpy as np
3 from sklearn.metrics import silhouette_score
4
5 # Test kilku warto ci eps
6 for eps in [0.2, 0.3, 0.5]:
7     db = DBSCAN(eps=eps, min_samples=5)
```

```

8 labels = db.fit_predict(X)
9 if len(set(labels)) > 1 and -1 not in set(labels):
10     score = silhouette_score(X, labels)
11     print(f"eps={eps}: silhouette score = {score:.2f}")
12
13 plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='plasma')
14 plt.title(f"DBSCAN (eps={eps})")
15 plt.show()

```

2.5 Agglomerative Clustering i dendrogram

```

1 from sklearn.cluster import AgglomerativeClustering
2 from scipy.cluster.hierarchy import dendrogram, linkage
3
4 linked = linkage(X, 'ward')
5 plt.figure(figsize=(10, 6))
6 dendrogram(linked)
7 plt.title("Dendrogram - Agglomerative Clustering")
8 plt.xlabel("Obiekty")
9 plt.ylabel("Odleg o ")
10 plt.show()
11
12 agg = AgglomerativeClustering(n_clusters=4)
13 labels_agg = agg.fit_predict(X)
14
15 plt.scatter(X[:, 0], X[:, 1], c=labels_agg, cmap='rainbow')
16 plt.title("Agglomerative Clustering")
17 plt.show()

```

2.6 Kryteria Calinskiego-Harabasz i Daviesa-Bouldina

```

1 from sklearn.metrics import calinski_harabasz_score, davies_bouldin_score
2
3 print("Calinski-Harabasz:", calinski_harabasz_score(X, y_kmeans))
4 print("Davies-Bouldin:", davies_bouldin_score(X, y_kmeans))

```

2.7 Ocena jakości grupowania

```
1 from sklearn.metrics import silhouette_score
2
3 score = silhouette_score(X, y_kmeans)
4 print(f"Wskaźnik Silhouette dla K-means: {score:.2f}")
```

3. Zadania do wykonania

1. Zmodyfikuj liczbę skupień w K-means i obserwuj wpływ na wynik.
2. Przetestuj różne wartości parametrów `eps` i `min_samples` w DBSCAN.
3. Porównaj wizualnie i numerycznie (np. silhouette score) wyniki dla trzech metod.
4. Zastosuj jedną z metod do danych rzeczywistych

Zbiory danych do analizy skupień (dane liczbowe)

Poniżej przedstawiono zbiory danych, które zawierają głównie dane liczbowe i mogą być bezpośrednio użyte do analizy skupień:

1. Iris: <https://archive.ics.uci.edu/ml/datasets/iris>
2. Wine: <https://archive.ics.uci.edu/ml/datasets/wine>
3. Seeds Dataset: <https://archive.ics.uci.edu/ml/datasets/seeds>
4. Wholesale Customers: <https://archive.ics.uci.edu/ml/datasets/wholesale+customers>
5. Credit Card Default: <https://archive.ics.uci.edu/ml/datasets/default+of+credit+card+clients>
6. Breast Cancer Wisconsin (diagnostic): [https://archive.ics.uci.edu/ml/datasets/Breast+Cancer+Wisconsin+\(Diagnostic\)](https://archive.ics.uci.edu/ml/datasets/Breast+Cancer+Wisconsin+(Diagnostic))
7. Digits: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_digits.html
8. Mall Customer Segmentation: <https://www.kaggle.com/datasets/vjchoudhary7/customer-segmentation-tutorial-in-python>

9. Countries of the World: <https://www.kaggle.com/datasets/fernando1/countries-of-the-world>
10. World Happiness: <https://www.kaggle.com/datasets/unsdsn/world-happiness>
11. FIFA Players: <https://www.kaggle.com/datasets/stefanoleone992/fifa-21-complete-player-dataset>
12. Air Quality Dataset: <https://archive.ics.uci.edu/ml/datasets/air+quality>
13. Concrete Compressive Strength: <https://archive.ics.uci.edu/ml/datasets/concrete+compressive+strength>
14. Parkinson's Telemonitoring: <https://archive.ics.uci.edu/ml/datasets/parkinsons+telemonitoring>
15. Statlog (Heart): [https://archive.ics.uci.edu/ml/datasets/statlog+\(heart\)](https://archive.ics.uci.edu/ml/datasets/statlog+(heart))